

ENTERPRISE ARCHITECTURE

Interview Handbook

Enterprise AI · Cloud Strategy · Security Architecture · Identity Design
Platform Engineering · Data Architecture · Agentic Systems · CTO-Level Strategy

Distinguished Architect Edition

Targeting: Microsoft · Google · Amazon · JPMorgan Chase · Goldman Sachs · Morgan Stanley
Capital One · Barclays · HSBC · Visa · Mastercard · ServiceNow · Salesforce · SAP

Roles: Enterprise Architect · Principal Architect · Chief Architect · Distinguished Architect
Head of Architecture · CTO · Chief AI Architect

TABLE OF CONTENTS

01 Enterprise Architecture Foundations

- TOGAF Deep-Dive
- Zachman Framework
- Gartner Continuous Architecture

02 Business Architecture

- Strategy Alignment
- Capability Mapping
- Org Design
- M&A; Scenarios

03 Executive Communication

- Board Scenarios
- CFO Conversations
- CEO Briefings
- Influence Without Authority

04 Security Architecture

- Zero Trust Design
- Machine Identity
- Secrets Management
- Compliance Frameworks

05 Identity Architecture

- Workforce Identity
- CIAM
- Machine Identity
- Agent Identity
- Authorization Models

06 Data Architecture

- Data Mesh vs Data Fabric
- 100 PB Modernisation
- Metadata Governance
- MDM

07 Platform Engineering

- IDP Design
- Developer Experience
- Platform Team Model
- Build vs Buy

08 Cloud Strategy

- Multi-Cloud vs Hybrid
- Cloud Sovereignty
- Landing Zone Design
- FinOps

09 Enterprise AI Architecture

- AI Operating Model
- Model Governance
- LLM Gateway
- AI Platform Design

10 Agentic Systems Architecture

- Agent Identity
- Agent Memory
- Human-in-the-Loop
- Agent Governance

11 Reliability Engineering

- AI-Specific SLOs
- Global Outage Recovery
- Cross-Region Resiliency

12 Banking Scenarios & Regulated Industries

- Global Banking AI Platform
- Multi-Jurisdictional Compliance
- Identity Convergence

13 Principal Architect Challenge Round

- Multi-Cloud Defence
- Autonomous Operations
- Core Banking Modernisation

14 Distinguished Architect Round

- First Principles
- Why Things Fail
- Architecture Philosophy

HOW TO USE THIS HANDBOOK

This handbook prepares architects at the Principal, Chief Architect, and Distinguished Architect levels for interviews at the world's leading technology companies and global financial institutions. Questions are designed to probe both technical depth and executive decision-making capability.

Each question includes:

- Level indicator (Principal / Chief Architect / Distinguished)
- Domain tags for quick filtering
- Hints — concepts to address before revealing the model answer
- Model answer — structured, opinionated, battle-tested patterns
- Follow-up challenge — the harder question that comes after
- Discussion areas — dimensions to explore in your response

For all questions, your answer should address these dimensions: business drivers and outcomes, architecture principles and tradeoffs, security and compliance implications, operational and reliability considerations, financial impact, governance and regulatory controls, and alternative approaches.

SECTION 01

ENTERPRISE ARCHITECTURE FOUNDATIONS

[TOGAF Deep-Dive] [Zachman Framework] [Gartner Continuous Architecture]

Q1 **DISTINGUISHED** [TOGAF] [Governance] [Cloud-Native]

Explain how TOGAF should be adapted for a cloud-native enterprise. Why do architecture governance programs fail?

DISCUSSION HINTS

- *ADM vs. agile delivery*
- *Governance overhead*
- *Outcome-driven architecture*

MODEL ANSWER

TOGAF's ADM was designed for a world of 3-year programmes and waterfall delivery. In a cloud-native enterprise, you must radically streamline it. Keep the Architecture Repository, Governance Framework, and Enterprise Continuum — discard the heavy phase-gate process in favour of continuous architecture practices.

Adaptation principles:

- Adopt Architecture Decision Records (ADRs) instead of heavy phase documents
- Embed architects in product teams — eliminate the ivory tower
- Automate governance via fitness functions in CI/CD pipelines
- Measure outcomes (DORA metrics, coupling indicators) not artefact completeness
- Tier governance: self-service for standard patterns, async review for new tech, synchronous ARB for cross-domain platforms only

Governance programs fail because: they become compliance theatre, focus on artefact production over outcomes, arrive too late in delivery cycles, and lack enforcement beyond 'approved with conditions'.

FOLLOW-UP CHALLENGE

How would you establish an Architecture Review Board at scale for 500 product teams?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q2 **CHIEF ARCHITECT** [ARB] [Scale] [Governance]

How would you establish an Architecture Review Board at scale?

DISCUSSION HINTS

- *Tiered governance*
- *Delegation model*
- *Async reviews*
- *Fitness functions*

MODEL ANSWER

At scale, a synchronous ARB reviewing every decision is a bottleneck, not an enabler.

Tiered model:

- Tier 1 (self-service): decisions within approved tech stack following reference architecture — no ARB required
- Tier 2 (async review): new technology adoption, reference architecture deviations — 5-day SLA, domain architect reviews ADR
- Tier 3 (synchronous ARB): cross-domain platforms, vendor contracts >\$1M, security architecture changes — quorum required

Make fitness functions do the heavy lifting: automated checks in CI/CD enforce coupling limits, encryption requirements, and API authentication standards. Reduce manual reviews by 60-70%.

Maintain a public Technology Radar (Adopt/Trial/Assess/Hold). Projects using 'Hold' technologies are automatically flagged without manual screening.

FOLLOW-UP CHALLENGE

You join a company where the ARB has a 3-month backlog. What are your first 90 days?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q3 **PRINCIPAL** [Zachman] [Taxonomy] [Framework Selection]

When would you choose Zachman over TOGAF? Where does Zachman commonly get misused?

DISCUSSION HINTS

- *Classification vs. methodology*
- *Enterprise taxonomy use case*
- *Documentation-heavy cultures*

MODEL ANSWER

Zachman is a classification framework — it defines what to document. TOGAF is a methodology — it defines how to develop architecture. They answer fundamentally different questions.

Choose Zachman when: building a comprehensive enterprise taxonomy (government, regulated industries), establishing documentation standards across federated units, or when audit traceability is the primary driver.

Common misuse: treating it as a delivery process, requiring all 36 cells before project approval, using it as a reporting tool, confusing classification schema with governance.

Best practice: use Zachman to define what good architecture artefacts look like, TOGAF governance constructs to manage how architecture gets done.

FOLLOW-UP CHALLENGE

Design an enterprise taxonomy for a global bank with 400 business capabilities.

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes

- Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

Q4 **PRINCIPAL** [Gartner] [Continuous Architecture] [Maturity]

How would you measure architecture maturity in an organisation that has never had a formal EA function?

DISCUSSION HINTS

- *Outcome vs. process compliance*
- *Dependency mapping diagnostic*
- *Quick wins*

MODEL ANSWER

Use a capability-outcome model rather than process-compliance checklist.

Five dimensions:

1. Visibility: Do decision-makers know the current-state landscape? (Shadow IT surface area, undocumented integrations)
2. Alignment: Are technology investments traceable to business capabilities?
3. Governance: Are standards enforceable or only aspirational?
4. Delivery Velocity: Time from idea to production — high coupling indicates low maturity
5. Technical Debt Ratio: % of engineering capacity consumed by remediation vs. innovation

Quick diagnostic: run a dependency mapping exercise across the top-20 systems. If no one can draw it accurately in a room, maturity is Level 1 regardless of what any framework assessment says.

FOLLOW-UP CHALLENGE

How do you communicate maturity gaps to a CFO who sees architecture investment as overhead?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

SECTION 02

BUSINESS ARCHITECTURE

[Strategy Alignment] [Capability Mapping] [Org Design] [M&A; Scenarios]

Q1 **CHIEF ARCHITECT** [Capability Map] [Banking] [Business Architecture]

Create a capability map for a global bank. How do business capabilities influence platform boundaries?

DISCUSSION HINTS

- Level 1/2/3 capabilities
- Platform boundaries follow capability ownership
- Conway's Law

MODEL ANSWER

A global bank capability map at Level 1:

- Customer Management: Onboarding, KYC/AML, Relationship Management, Customer Analytics
- Payments: Domestic Payments, Cross-Border Payments, Real-Time Payments, Treasury Payments
- Lending: Credit Origination, Credit Decisioning, Loan Servicing, Collections
- Deposits & Accounts: Account Management, Interest Calculation, Statements
- Risk & Compliance: Credit Risk, Market Risk, Operational Risk, Regulatory Reporting
- Finance: GL, Financial Reporting, FinOps, Transfer Pricing
- Technology: Platform Engineering, Security, Data, Integration

Capability boundaries should define platform boundaries (Conway's Law applied intentionally). Each Level-1 capability domain becomes a stream-aligned team boundary. Shared capabilities (Identity, Data, Payments Rail) become platform teams. This prevents the accidental coupling that emerges when teams share code without shared ownership.

FOLLOW-UP CHALLENGE

The Payments platform team wants to own fraud detection. The Risk team also claims it. How do you resolve the capability ownership conflict?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q2 **DISTINGUISHED** [Conway's Law] [Team Topologies] [M&A;]

Explain value streams versus organisational structures. How does Conway's Law apply to a post-merger integration?

DISCUSSION HINTS

- Value stream cuts across org silos
- Inverse Conway Maneuver
- Team cognitive load

MODEL ANSWER

Value streams describe end-to-end flows of value to the customer, cutting across organisational silos. Org structures describe reporting lines. The mismatch between them is the primary source of handoff waste and coordination overhead.

Conway's Law: any organisation designing a system produces a design whose structure mirrors its communication structure. In M&A, two banks with different org structures will produce architectures that reflect both — creating seams at integration points.

Inverse Conway Maneuver for M&A:

1. Define the target architecture first (domain boundaries: Accounts, Payments, Lending, Risk)
2. Map each domain to a combined team drawn from both institutions
3. Protect the acquired fintech's product team structure — their velocity is the acquisition value
4. Integrate at the API and identity layer first — avoid premature database mergers

The architecture will converge to the team structure within 18-24 months. Design the teams deliberately.

FOLLOW-UP CHALLENGE

The acquired fintech's engineers threaten to resign if you impose the bank's governance model. How do you handle it?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q3 CHIEF ARCHITECT [Strategy] [Roadmap] [Transformation]

Design a 5-year modernisation roadmap for an estate of 4,000 applications.

DISCUSSION HINTS

- Application portfolio rationalisation
- Wave-based sequencing
- Funding model
- Dependency mapping

MODEL ANSWER

Phase 0 (months 1-3) — Portfolio Assessment:

Classify all 4,000 applications by: business value (revenue/cost/risk), technical health (debt, supportability), strategic fit, cloud-readiness. Apply TIME model: Tolerate / Invest / Migrate / Eliminate.

Phase 1 (Year 1) — Foundation:

- Eliminate: sunset 30-40% of applications (duplicates, abandoned, replaceable by SaaS)
- Establish platform foundations: identity, observability, CI/CD, landing zones
- Quick wins: lift-and-shift 200 low-complexity apps to reduce data centre footprint

Phase 2 (Years 2-3) — Differentiation:

- Modernise high-value, high-debt core systems using strangler fig pattern
- Invest: re-platform 500 strategic apps to cloud-native

- Decommission legacy infrastructure freed by migrations

Phase 3 (Years 4-5) — Innovation:

- AI-enabled platforms, event-driven architecture at scale
- Residual legacy estate <5% of original

Funding model: fund from data centre savings (Year 1 savings fund Year 2 investment). Create a Transformation Office with P&L; visibility.

FOLLOW-UP CHALLENGE

How would you prioritise modernisation candidates when business units disagree on priorities?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

SECTION 03

EXECUTIVE COMMUNICATION

[Board Scenarios] [CFO Conversations] [CEO Briefings] [Influence Without Authority]

Q1 **CHIEF ARCHITECT** [AI Investment] [CFO] [Business Case]

Present a \$100M AI investment proposal to a CFO.

DISCUSSION HINTS

- ROI framing
- Risk-adjusted returns
- Phased commitment
- Competitive necessity

MODEL ANSWER

CFO framing — lead with numbers, not technology:

'We are asking for \$100M over 3 years. Here is the return case:

Year 1 (\$30M): Internal productivity AI — copilots for 10,000 engineers and analysts. Conservative estimate: 20% productivity gain = \$40M in recovered capacity annually. Payback: 9 months.

Year 2 (\$40M): Customer-facing AI — AI-driven loan origination, personalised advice, intelligent document processing. Expected revenue uplift: \$80M annually from faster decisioning and reduced drop-off. Risk: regulatory approval required (scoped).

Year 3 (\$30M): Autonomous operations — AI agents for compliance reporting, reconciliation, fraud investigation. Cost reduction target: \$60M annually from automation of high-volume manual processes.

Total 3-year return: \$540M against \$100M investment. NPV positive by Month 18.

Risk: not investing. Three competitors have already announced AI programmes. The cost of catching up in Year 4 is estimated at \$300M+ and 2 years of lost market position.'

FOLLOW-UP CHALLENGE

The CFO wants to fund \$30M but not the full \$100M. How do you re-sequence?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

Q2 **CHIEF ARCHITECT** [Technical Debt] [CFO] [Financial Metaphor]

How do you communicate technical debt to executives?

DISCUSSION HINTS

- Financial debt analogy
- Carrying cost

- *Opportunity cost*
- *Debt ledger*

MODEL ANSWER

Technical debt is exactly like financial debt — it has principal and an interest rate.

The principal: deferred work — code not refactored, integrations not modernised, tests not written.

The interest: the extra cost paid every day — slower delivery, higher incident rates, more engineers needed to change a system that was never designed to change.

For a CFO: 'We currently spend approximately 25% of engineering capacity on interest payments — workarounds, incident response, re-work on brittle systems. That is \$X per year in lost productivity. The \$Y remediation investment pays back in Z years through recovered capacity and eliminates the compounding risk.'

Bring a debt ledger: a one-page visual showing debt concentration by system, associated incident rates, and team velocity impact. CFOs make decisions when they can see the financial exposure clearly.

FOLLOW-UP CHALLENGE

The CFO says 'we funded this 3 years ago and still have the same problems.' How do you respond honestly?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q3 **DISTINGUISHED** [Influence] [Leadership] [Stakeholder Management]

How do you influence without authority as a Chief Architect?

DISCUSSION HINTS

- *Credibility*
- *Coalition building*
- *Making the safe choice the easy choice*
- *Architecture as product*

MODEL ANSWER

Authority is a poor substitute for credibility. Chief Architects who rely on authority get compliance; those who build credibility get commitment.

Five levers:

1. Expertise: Be the person who has thought more carefully about the tradeoffs than anyone else in the room. Your opinion carries weight because of demonstrated depth.
2. Evidence: Decisions backed by data, case studies, and fitness function measurements beat opinions every time.
3. Framing: Translate architecture into the language of your audience — financial risk for CFOs, competitive position for CEOs, developer velocity for CTOs.
4. Make the right path easy: Golden paths, reference architectures, and IDP templates make the architecturally correct choice the path of least resistance. Engineers follow paved roads.

5. Build coalitions: Find the influential principal engineers in each domain and invest in them. Architecture scales through the community, not through mandates.

FOLLOW-UP CHALLENGE

A senior VP is sponsoring an architecture decision you believe is technically wrong. How do you handle it?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q4 **DISTINGUISHED** [Zero Trust] [Board] [Executive Communication]

Present Zero Trust Architecture to a Board of Directors in 5 minutes.

DISCUSSION HINTS

- *No jargon*
- *Business risk frame*
- *Regulatory necessity*
- *Concrete outcomes*

MODEL ANSWER

'Our traditional security model assumed that once inside our network, you were trusted. It is the equivalent of a building with a strong front door but no locks on internal rooms.

Zero Trust means: no one is trusted by default — not employees, not vendors, not systems — until they prove who they are, on every request, every time.

Why now: 68% of breaches in financial services involve compromised insider credentials. The perimeter is not the threat boundary. Regulators (OCC, FRB, ECB) are explicitly requiring Zero Trust capability in 2025-26 examinations. Three of our top-five competitors have deployed it.

What we are building: continuous verification of every user, device, and system; no implicit trust between internal applications; all sensitive data encrypted even on our internal network.

Investment: \$X over 3 years. Risk reduction: 70% reduction in lateral movement attack surface. Regulatory outcome: clean examination posture on security architecture.'

FOLLOW-UP CHALLENGE

A board member asks: 'We have invested in firewalls for 20 years. Why didn't that work?'

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

SECTION 04

SECURITY ARCHITECTURE

[Zero Trust Design] [Machine Identity] [Secrets Management] [Compliance Frameworks]

Q1 **DISTINGUISHED** [Zero Trust] [Multi-Cloud] [Design]

Design Zero Trust Architecture for a multi-cloud enterprise with 100,000 employees and 2,000 applications.

DISCUSSION HINTS

- Identity as control plane
- Microsegmentation
- Device posture
- mTLS service mesh

MODEL ANSWER

Five control planes:

1. Identity Control Plane: Entra ID for workforce (OIDC/SAML/SCIM), dedicated CIAM for customers, SPIFFE/SPIRE for workloads (X.509 SVIDs), CyberArk PAM for privileged access.
2. Device Trust: MDM enrollment (Intune/Jamf), continuous device posture scoring, Conditional Access — unmanaged device = read-only + DLP enforcement.
3. Network: ZTNA replaces VPN (Zscaler/Cloudflare Access), service mesh with mTLS (Istio) for east-west traffic, no flat production network segments.
4. Application: OIDC with short-lived tokens (15-min expiry), step-up authentication for sensitive operations, ABAC policies for data-sensitive applications.
5. Data: DSPM for classification and inventory, DLP on egress paths, field-level encryption for PII/PCI.

Unified policy engine (OPA/Cedar) manages authorisation policy consistently across all planes.

FOLLOW-UP CHALLENGE

A vendor needs access to 500 internal applications for a 2-year engagement. Design the access model.

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

Q2 **PRINCIPAL** [Machine Identity] [SPIFFE] [Workload Identity]

How should machine identities be governed at enterprise scale?

DISCUSSION HINTS

- SPIFFE/SPIRE

- *Workload Identity Federation*
- *Short-lived credentials*
- *Zero static keys*

MODEL ANSWER

Machine identity is the fastest-growing attack surface in enterprise environments. Static service account keys and long-lived certificates are the primary vulnerability.

Governance model:

- Every workload (pod, function, VM, pipeline) gets a cryptographic identity (SPIFFE SVID — X.509 or JWT)
- SVIDs are short-lived (1-hour TTL) and auto-rotated by the SPIRE agent
- No static cloud provider keys in production — Workload Identity Federation maps SPIFFE identities to cloud IAM roles (AWS IRSA, Azure Workload Identity, GCP Workload Identity)
- Machine identity registry: every identity catalogued with owner, purpose, and declared scope
- Certificate inventory: all TLS certificates tracked with expiry alerts at 60/30/7 days
- CI/CD pipelines: OIDC tokens from GitHub Actions/GitLab authenticate to cloud resources — zero long-lived secrets in code repositories

Governance controls: quarterly rotation audits, automated secret scanning in repos, SIEM alerts on anomalous machine identity usage.

FOLLOW-UP CHALLENGE

A developer hard-codes an AWS access key in a GitHub repository. Walk through your incident response.

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q3 **PRINCIPAL** [Secrets Management] [Vault] [Multi-Cloud]

Describe an enterprise secrets management architecture for a multi-cloud bank.

DISCUSSION HINTS

- *HashiCorp Vault as federation layer*
- *Dynamic credentials*
- *Break-glass procedures*
- *Audit trail*

MODEL ANSWER

Architecture decision: HashiCorp Vault Enterprise as federation layer over cloud-native KMS.

Rationale: cloud-native secrets managers (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager) are excellent within their cloud but create a split management problem in multi-cloud. Vault provides a unified secrets API and audit log across all three.

Design:

- Vault Enterprise Cluster: 3-region active-passive, auto-unseal via cloud KMS (BYOK)
- Auth Methods: AWS IAM, Azure Managed Identity, GCP SA, Kubernetes (per cluster), OIDC
- Secret Engines: Database (dynamic credentials 15-min TTL), PKI (on-demand TLS certs), Transit (encryption-as-a-service), KV v2 (version-controlled static secrets)

- Audit: all access events streamed to SIEM (Splunk/Sentinel)

Key principles:

- No static credentials in production — all secrets dynamic with TTL
- Zero human access to production secrets — all via workload identity
- Break-glass: time-limited, dual-approval, fully audited emergency access
- Vault Agent sidecars in Kubernetes inject secrets at runtime — no secrets in environment variables

FOLLOW-UP CHALLENGE

Your PKI intermediate CA certificate is 2 weeks from expiry and was not in the rotation schedule. What do you do?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

SECTION 05

IDENTITY ARCHITECTURE

[Workforce Identity] [CIAM] [Machine Identity] [Agent Identity] [Authorization Models]

Q1 **DISTINGUISHED** [Identity] [Scale] [Multi-Cloud]

Design identity architecture for 500,000 employees, 50 million customers, and 20,000 applications.

DISCUSSION HINTS

- Separation of workforce vs. CIAM
- Federation topology
- Token strategy
- Performance at scale

MODEL ANSWER

Three identity planes — never co-mingle them:

1. Workforce Identity (500K employees):

- Primary IdP: Microsoft Entra ID (global, HA)
- Tier-1 apps: OIDC/OAuth2 direct; Tier-2 legacy: SAML 2.0; Tier-3 on-prem: Kerberos via Entra DS
- MFA: FIDO2/Passkeys for privileged roles, TOTP for standard users
- Tokens: 15-minute access tokens, rotating refresh tokens — zero long-lived bearer tokens

2. Customer Identity (50M customers):

- Dedicated CIAM platform (Okta CIC, Auth0, Ping, or ForgeRock) — NOT Entra
- OIDC for all digital channels; progressive profiling; Passkeys for step-up
- Data residency compliance per jurisdiction (EU data stays EU)

3. Machine Identity (workloads):

- SPIFFE/SPIRE: every pod/function gets X.509 SVID
- Workload Identity Federation: eliminate cloud service account key sprawl
- OIDC for CI/CD pipeline authentication to cloud resources

Federation topology: Entra as hub, SaaS as spokes (OIDC preferred, SAML for legacy). External B2B: OIDC/SAML federation with trust verification. Central token validation service inline with API Gateway for revocation propagation.

FOLLOW-UP CHALLENGE

One of your top SaaS vendors requires SAML only but your strategy is OIDC-first. How do you handle the exception?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

Q2 **PRINCIPAL** [OAuth2] [OBO Flow] [Delegation]

Explain the OAuth2 On-Behalf-Of (OBO) flow. How would ServiceNow user delegation work using it?

DISCUSSION HINTS

- RFC 7523 JWT Bearer Grant
- Downstream API calls
- Token exchange
- ServiceNow MID server

MODEL ANSWER

OBO Flow (On-Behalf-Of / RFC 7523):

Problem: Service A receives a user token and needs to call Service B on behalf of that user, but Service B requires proof of the user's identity, not just Service A's identity.

Flow:

1. User authenticates to App, receives access token for Service A
2. Service A presents user's token + its own client credentials to the token endpoint
3. Token endpoint validates both, issues a new access token scoped for Service B with subject = original user
4. Service A calls Service B with the new delegated token

ServiceNow delegation example:

- User logs into ServiceNow with their Entra ID token
- ServiceNow Flow Designer automation calls an external HR API on behalf of the user
- ServiceNow presents user token + its app registration to Entra to exchange for HR API-scoped token
- HR API receives a token with: aud=HR API, sub=original user, act=ServiceNow app
- Full audit trail: every HR API call is attributable to the originating user, not a service account

Critical: the downstream token scope must be a subset of the original token — OBO cannot escalate privileges.

FOLLOW-UP CHALLENGE

Design the authorisation model for an AI agent that acts on behalf of a user and orchestrates sub-agents across 50 enterprise APIs.

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q3 **PRINCIPAL** [ABAC] [RBAC] [Authorization] [Scale]

When do you choose ABAC over RBAC? How do you implement ABAC at enterprise scale without policy explosion?

DISCUSSION HINTS

- Context-sensitivity
- OPA/Cedar
- Policy-as-code

■ Attribute taxonomy

MODEL ANSWER

Decision framework:

- RBAC: use when access is determined by who you are (job title, department, group). Simple, auditable.
- ABAC: use when access depends on context — what resource, from where, at what time, under what risk level
- ReBAC: use when access depends on relationships — 'can this manager see this employee's record?'

ABAC is unavoidable in banking when:

- A relationship manager can see only their own clients' records
- Export is allowed on managed devices during business hours, not on weekends
- Data residency enforcement — EU users cannot access records stored outside EU

Taming ABAC at scale:

1. Policy-as-code: OPA or AWS Cedar — version-controlled, testable, auditable
2. Policy catalog: centralise definitions; apps reference policies by name
3. Decision logging: every authorisation decision logged with full context
4. Performance: OPA <5ms P99; cache attribute bundles; pre-compute slow attributes (org hierarchy)
5. Policy testing pipeline: regression test suite on every policy change before production
6. Attribute taxonomy governance: standardised attribute names prevent policy proliferation

FOLLOW-UP CHALLENGE

An AI agent acts on behalf of a user — can the agent be granted MORE permissions than the delegating user? Justify your answer architecturally.

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

SECTION 06

DATA ARCHITECTURE

[Data Mesh vs Data Fabric] [100 PB Modernisation] [Metadata Governance] [MDM]

Q1 **DISTINGUISHED** [Data Mesh] [Data Fabric] [Architecture Choice]

Data Mesh versus Data Fabric — how do you choose, and where does each fail?

DISCUSSION HINTS

- Organisational maturity
- Decentralised ownership vs. unified management
- Governance model

MODEL ANSWER

These are not competing technologies — they are different organisational paradigms.

Data Mesh: decentralises data ownership to domain teams. Each domain owns, produces, and serves its data as a product. Requires: strong data product culture, mature domain teams, federated governance model.

Fails when: domain teams lack data engineering skills, governance is too weak to enforce interoperability standards, or the organisation is too centralised to enable domain autonomy.

Data Fabric: centralised logical layer that connects distributed data sources with unified metadata, lineage, and access control. Automated discovery and integration via AI/ML cataloguing.

Fails when: source systems have poor data quality, metadata management is manual and inconsistent, or the central team becomes a bottleneck.

Choose Data Mesh when: large organisation, mature product teams, willing to invest in data product discipline.

Choose Data Fabric when: heterogeneous legacy estate, need for unified governance without reorganisation, data integration is the primary problem.

Pragmatic approach for most enterprises: implement Data Fabric as the governance and integration layer, incrementally apply Data Mesh principles to domains as they mature.

FOLLOW-UP CHALLENGE

How do you implement Data Mesh governance without creating a central bottleneck?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

Q2 **CHIEF ARCHITECT** [Data Modernisation] [100 PB] [Strategy]

How would you modernise a 100 PB enterprise data estate?

DISCUSSION HINTS

- Hot/warm/cold tiering

- *Open table formats*
- *Data classification first*
- *Incremental migration*

MODEL ANSWER

A 100 PB estate cannot be migrated — it must be evolved in place with selective migration.

Phase 1 — Classify and Tier (months 1-4):

- Classify by access frequency: hot (daily), warm (monthly), cold (archival)
- Classify by sensitivity: public, internal, confidential, restricted
- Identify data that can be purged (expired retention, duplicates, orphaned) — often 20-30% of estate

Phase 2 — Open Format Migration (Year 1):

- Migrate to open table formats (Apache Iceberg) — enables multi-engine querying
- Lakehouse pattern: single copy of truth queryable by SQL engines, ML frameworks, and streaming systems
- Implement data tiering: hot on cloud object storage + analytics engine, cold on glacier-class

Phase 3 — Domain Data Products (Years 2-3):

- Apply Data Mesh principles to highest-value domains first
- Each domain publishes certified data products with SLAs

Phase 4 — AI-Ready Data Platform (Years 3-5):

- Vector stores for unstructured data, feature stores for ML, real-time streaming for AI inference

Governance throughout: unified metadata catalog (Apache Atlas/DataHub), automated lineage, data quality SLAs.

FOLLOW-UP CHALLENGE

Regulators require you to demonstrate complete data lineage for all PII within 48 hours of a subject access request. How does your architecture support this?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q3 **PRINCIPAL** [Metadata] [Data Governance] [Lineage]

Design an enterprise metadata governance strategy.

DISCUSSION HINTS

- *Active vs. passive metadata*
- *DataHub/Collibra/Atlas*
- *Lineage capture*
- *Data stewardship model*

MODEL ANSWER

Metadata governance is the foundation of every data quality, compliance, and AI programme. Without it, you are building on sand.

Four metadata types to govern:

1. Technical metadata: schemas, data types, table structures — captured automatically from source systems
2. Operational metadata: data freshness, volume, pipeline execution — captured from ETL/streaming logs
3. Business metadata: business definitions, data owners, data stewards — curated by domain teams
4. Governance metadata: classification labels, retention policies, consent records — enforced via policy engine

Platform: unified data catalog (DataHub, Collibra, or Apache Atlas) as the single source of metadata truth.

Lineage strategy:

- Automated lineage capture from ETL tools (Spark, dbt, Informatica) via OpenLineage standard
- Column-level lineage for PII traceability (GDPR/CCPA subject access requests)
- Impact analysis: before any schema change, catalog shows all downstream consumers

Stewardship model: federated — domain data stewards own business metadata, central data governance team sets standards and resolves conflicts.

FOLLOW-UP CHALLENGE

A data scientist deploys a model trained on data with incorrect lineage metadata. How does your governance architecture detect and respond?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

SECTION 07

PLATFORM ENGINEERING

[IDP Design] [Developer Experience] [Platform Team Model] [Build vs Buy]

Q1 **CHIEF ARCHITECT** [Platform Team] [When to Create] [Team Topologies]

When should a platform team exist? How do you measure platform adoption?

DISCUSSION HINTS

- Cognitive load threshold
- Platform as product
- DORA metrics
- NPS for platforms

MODEL ANSWER

A platform team is justified when: the cognitive load of infrastructure, tooling, and compliance would otherwise consume >30% of stream-aligned team capacity, AND when the problem is common across 3+ teams.

The platform team's job is to reduce cognitive load on stream-aligned teams — not to control them.

When NOT to create a platform team: when you have fewer than 50 engineers (overhead exceeds benefit), when the platform would be more restrictive than enabling, or when stream-aligned teams could self-serve with minimal investment in tooling and documentation.

Measuring platform adoption:

- Adoption rate: % of eligible teams using the platform capability vs. rolling their own
- Time-to-first-deploy: median time for a new service to reach production using the platform (target: <1 day)
- Developer NPS: quarterly survey — 'Would you recommend the platform to another team?'
- DORA metrics per team: deployment frequency, lead time, MTTR, change failure rate
- Support ticket volume: platform maturity should reduce, not increase, support load

If adoption is low, do not mandate — investigate why the platform is not the path of least resistance.

FOLLOW-UP CHALLENGE

40% of your engineers are bypassing the IDP and using cloud consoles directly. What do you do?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

Q2 **DISTINGUISHED** [Build vs Buy] [Strategy] [Platform Economics]

Build versus Buy tradeoff analysis for a core banking platform. How do you decide?

DISCUSSION HINTS

- Differentiation vs. commodity
- Total cost of ownership

- *Vendor lock-in*
- *Strategic capability*

MODEL ANSWER

The decision framework: build what differentiates, buy what commoditises.

For core banking specifically — this is one of the most consequential decisions in financial services:

Buy when:

- The capability is not a source of competitive differentiation
- The vendor has deep regulatory expertise (Basel III, IFRS 9, PSD2, local regulations)
- Time-to-market is critical and internal build would take 3+ years
- Total cost of ownership (TCO) of build exceeds buy over 7-year horizon

Build when:

- The capability IS your competitive moat (proprietary risk models, unique customer experience)
- Vendor products cannot support your business model (neobank, embedded finance)
- Vendor lock-in risk exceeds build risk over strategic horizon
- You have the engineering talent and organisational patience to build and operate it

For most banks: buy commodity cores (ledger, settlement, regulatory reporting), build differentiating layers (customer experience, AI decisioning, distribution).

TCO model must include: licence, implementation, integration, talent, upgrade cycles, migration optionality cost (exit cost from vendor).

FOLLOW-UP CHALLENGE

Your chosen core banking vendor is acquired by a competitor. What is your architectural response?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

SECTION 08

CLOUD STRATEGY

[Multi-Cloud vs Hybrid] [Cloud Sovereignty] [Landing Zone Design] [FinOps]

Q1 **DISTINGUISHED** [Multi-Cloud] [Hybrid] [Strategy]

Multi-cloud versus hybrid cloud — how do you choose, and what does intentional multi-cloud look like?

DISCUSSION HINTS

- *Accidental vs. intentional*
- *Regulatory sovereignty*
- *Differentiated cloud capabilities*
- *Complexity cost*

MODEL ANSWER

Most enterprises are already multi-cloud accidentally — different teams chose different providers. This is the worst outcome: all the complexity with none of the strategic benefit.

Intentional multi-cloud: each cloud is chosen for its differentiated capability.

Example allocation for a global bank:

- Azure: AI/ML workloads (Azure OpenAI integration, Entra ID native), Office 365 integration
- AWS: core banking compute (EC2 bare-metal for latency-sensitive), financial services ecosystem
- GCP: analytics and data platform (BigQuery, Vertex AI, Dataplex governance)
- On-premises/sovereign cloud: ultra-sensitive data, jurisdictions with data residency requirements

Hybrid cloud: production workloads span on-premises and cloud, unified by a management plane (Azure Arc, AWS Outposts, Anthos).

Architecture principles for intentional multi-cloud:

- Cloud-agnostic data layer: open formats (Iceberg/Delta/Parquet), portable across clouds
- Federated identity: Workload Identity Federation eliminates cloud-specific key sprawl
- Infrastructure-as-code abstraction: Terraform modules, not cloud-specific CDK
- Container portability: Kubernetes workloads runnable on AKS/EKS/GKE

FOLLOW-UP CHALLENGE

Your CISO argues that multi-cloud increases attack surface. How do you respond?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

Q2 **PRINCIPAL** [Cloud Sovereignty] [Regulatory] [Data Residency]

Design a cloud sovereignty architecture for a bank operating in the EU, India, and Singapore.

DISCUSSION HINTS

- DORA
- PDPA
- MAS TRM
- Data localisation
- Sovereign cloud options

MODEL ANSWER

Cloud sovereignty requires that data, operations, and the ability to audit all remain within jurisdictional control.

EU (DORA + GDPR):

- Data must not leave EU: Azure EU Data Boundary or AWS EU Sovereign Cloud
- DORA requires: operational resilience testing, incident reporting <4 hours, ICT vendor risk management
- Key management: customer-managed keys (BYOK) in EU HSMs — cloud provider has no key access
- Concentration risk: no single cloud provider >X% of critical functions (DORA principle)

India (RBI guidelines + PDPA):

- Payment data must be stored exclusively in India — no cross-border transfer
- Options: AWS Mumbai/Hyderabad regions, Azure India Central, or on-premises
- RBI mandates audit access and examination rights over cloud infrastructure

Singapore (MAS TRM):

- MAS Technology Risk Management guidelines require: concentration risk limits, outsourcing controls, exit planning
- Data classification determines residency requirement — SingPass-linked data stays in Singapore

Architecture pattern: regional landing zones with sovereign controls, unified management plane, cross-region replication prohibited for regulated data classes.

FOLLOW-UP CHALLENGE

A single cloud provider suffers a regional outage affecting your EU operations. How does your sovereign architecture ensure continuity?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q3 CHIEF ARCHITECT [Landing Zone] [Global] [Governance]

Design a global landing zone architecture for a Fortune 100 bank.

DISCUSSION HINTS

- Management hierarchy
- Network topology
- Security baseline
- Policy-as-code

MODEL ANSWER

Landing zone design principles: consistent, governed, and automatable foundation for every workload.

Management hierarchy (Azure example — equivalent exists on AWS/GCP):

Root Management Group

■ ■ ■ ■ Platform (Identity, Connectivity, Management subscriptions)

■ ■ ■ ■ Landing Zones

■ ■ ■ ■ Production (Corp, Online, Confidential)

■ ■ ■ ■ Non-Production (Dev, Test, Staging)

■ ■ ■ ■ Sovereign (EU, APAC, regulated workloads)

■ ■ ■ ■ Sandbox (unmanaged experimentation)

Network topology: Hub-and-Spoke or Azure Virtual WAN. Central hub contains: firewall, DNS, VPN/ExpressRoute gateway. Spokes: workload VNets peered to hub. No spoke-to-spoke traffic without hub traversal.

Security baseline (enforced via Azure Policy / AWS SCP / GCP Org Policy):

- All storage encrypted at rest
- Public internet access blocked by default — exceptions require explicit approval
- All resources tagged with owner, environment, data classification
- Diagnostic logs streamed to central SIEM

FinOps: cost allocation by subscription maps to product P&L.; Budget alerts at 80%/100% thresholds.

FOLLOW-UP CHALLENGE

A business unit deploys directly to cloud bypassing the landing zone. How does your architecture detect and remediate this?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

SECTION 09

ENTERPRISE AI ARCHITECTURE

[AI Operating Model] [Model Governance] [LLM Gateway] [AI Platform Design]

Q1 **DISTINGUISHED** [AI Operating Model] [Governance] [SR 11-7]

Design an AI operating model and model governance lifecycle for a global bank.

DISCUSSION HINTS

- SR 11-7 compliance
- Risk tiering
- Validation framework
- Human-in-the-loop

MODEL ANSWER

AI Operating Model — three pillars: Platform, Process, People.

Platform (see AI Gateway architecture question).

Process — Model Governance Lifecycle:

1. Intake: model proposal with use case, risk classification, data requirements
2. Risk Tiering (SR 11-7): Tier 1 (low, non-consequential) to Tier 4 (high, consequential decisions affecting consumers)
3. Development: in approved AI Workbench with pre-approved libraries and data access controls
4. Validation: independent model validation team reviews Tier 3-4 models — accuracy, bias, fairness, adversarial robustness
5. Approval: Model Risk Committee approval for Tier 3-4. Tier 1-2 self-certified by model owner.
6. Production: deployed via ML platform with full monitoring and drift detection
7. Monitoring: continuous performance monitoring, fairness monitoring, distribution drift alerts
8. Review: annual review for all production models; immediate review on material performance degradation
9. Retirement: controlled decommission with documentation of business impact

People:

- Chief AI Officer: strategy, governance, regulatory relationships
- AI Platform team: platform engineering and reliability
- Model Risk team: independent validation (reports to CRO, not CTO)
- Domain AI teams: model development and ownership
- AI Ethics council: fairness and responsible AI oversight

FOLLOW-UP CHALLENGE

A regulator asks you to demonstrate that your AI credit decisioning model is free from discriminatory bias. Walk them through your governance architecture.

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics

- Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

Q2 CHIEF ARCHITECT [AI Gateway] [LLM] [Architecture]

Design an enterprise AI gateway architecture for a bank with 10,000 AI developers.

DISCUSSION HINTS

- PII scrubbing
- Model routing
- Cost attribution
- Rate limiting
- Audit trail

MODEL ANSWER

The AI Gateway is the control plane for all LLM traffic — internal and external.

Core functions:

1. Authentication/Authorisation: every request authenticated (OIDC/API key), authorised against model access policy
2. PII Detection and Scrubbing: presidio-based PII detection on all prompts before external model calls; block or pseudonymise based on data classification policy
3. Content Safety: pre-flight checks on prompts for policy violations; post-flight checks on responses
4. Model Routing: route to optimal model based on cost, latency, capability tier, and task type
5. Rate Limiting: per-team, per-application token budgets; cost alerts at 80% threshold
6. Cost Attribution: all token consumption attributed to cost centre, enabling FinOps chargeback
7. Audit Logging: immutable log of all prompts and responses (WORM storage) for SR 11-7 and regulatory examination
8. Caching: semantic caching of identical/similar prompts to reduce cost and latency

Model support: OpenAI, Azure OpenAI, Anthropic, AWS Bedrock, Vertex AI — unified interface hides provider.

Deployment: active-active multi-region, <10ms gateway overhead P99, circuit breaker per model provider, automatic failover to secondary provider.

FOLLOW-UP CHALLENGE

A developer inadvertently sends a prompt containing 10,000 customer records to an external model provider. What happens in your architecture?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

SECTION 10

AGENTIC SYSTEMS ARCHITECTURE

[Agent Identity] [Agent Memory] [Human-in-the-Loop] [Agent Governance]

Q1 **DISTINGUISHED** [Agent Identity] [Authorization] [RFC 8693]

Design agent identity and authorisation for autonomous AI agents acting on behalf of users in a regulated bank.

DISCUSSION HINTS

- SPIFFE for agent identity
- Token Exchange RFC 8693
- Scope downscoping
- Blast radius limits

MODEL ANSWER

Core principle: least-privilege delegation, not impersonation. An agent should never have MORE access than the delegating user.

Agent Identity:

- Every agent registered in an agent registry before deployment: name, owner, purpose, risk tier, declared tool scope
- Agent receives SPIFFE SVID (X.509 machine identity) — no anonymous agents permitted

Delegation Model — RFC 8693 Token Exchange:

1. User authenticates, receives user access token
2. Agent requests delegation via Token Exchange endpoint
3. Delegated token: subject=user, actor=agent, scope=requested subset of user's scope
4. Agent calls APIs with delegated token — scope enforced by authorisation server
5. All API calls audited: actor=agent, on-behalf-of=user

Sub-agent orchestration: parent agent delegates to sub-agents via same Token Exchange. Scope can only narrow — never widen — down the delegation chain.

Runtime controls:

- Blast radius limits: max N actions, max \$X spend, max T minutes per session before human checkpoint
- Irreversible actions require confirmation token (wire transfer, record deletion, external communication)
- Anomaly detection: prompt injection detection, scope escalation alerts, unusual tool invocation patterns
- Kill switch: disable all agents by risk tier within 60 seconds globally

FOLLOW-UP CHALLENGE

An agent gets prompt-injected via a malicious document and attempts to initiate a wire transfer. Walk through your containment architecture.

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics

- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q2 **PRINCIPAL** [Agent Memory] [Architecture] [Privacy]

Design an agent memory architecture for enterprise AI agents that handles both short-term and long-term memory with privacy controls.

DISCUSSION HINTS

- Working memory vs. episodic vs. semantic
- Vector stores
- Data retention
- User consent

MODEL ANSWER

Four memory types for enterprise agents:

1. Working Memory (in-context, session-scoped):

- The agent's active context window for the current task
- Cleared at session end — no persistence by default
- Contains: task instructions, retrieved context, tool outputs, intermediate reasoning

2. Episodic Memory (user-scoped, short-term):

- Memories of specific past interactions with a user
- Stored in vector DB with user ID as namespace
- Retention: user-configurable, default 90 days, auto-purged
- Privacy control: user can view, edit, and delete their episodic memories

3. Semantic Memory (shared, long-term):

- Knowledge about the enterprise: policies, procedures, product information
- Curated knowledge base, not derived from user interactions
- Access-controlled: different semantic memory namespaces per security classification

4. Procedural Memory (agent skills):

- Learned tool usage patterns, successful task strategies
- Version-controlled, governed change process

Privacy architecture:

- No customer PII in long-term memory without explicit consent
- All memory writes logged for audit
- GDPR right-to-erasure: user memory deletion cascades across all memory types
- Encryption at rest with user-specific key derivation

FOLLOW-UP CHALLENGE

An agent's episodic memory is poisoned via prompt injection — a bad actor has inserted false memories about a customer's credit profile. How do you detect and remediate?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q3 **CHIEF ARCHITECT** [Human-in-the-Loop] [Governance] [Risk Tiering]

Design a human-in-the-loop governance model for AI agents in a regulated financial institution.

DISCUSSION HINTS

- Risk-tiered oversight
- Consequential decision definition
- Approval workflows
- Override audit trail

MODEL ANSWER

Human-in-the-loop is not a binary choice — it is a risk-tiered spectrum.

Tier 1 — Fully Autonomous (Low Risk, Reversible):

- Examples: summarisation, information retrieval, internal report drafting
- Human oversight: output reviewed after the fact via audit logs
- No blocking approval required

Tier 2 — Human-on-the-Loop (Medium Risk, Reversible Writes):

- Examples: customer communication drafts, CRM updates, document generation
- Human reviews agent output before it is committed or sent
- Approval window: 24 hours; auto-cancelled if not approved

Tier 3 — Human-in-the-Loop (High Risk, Consequential):

- Examples: credit decisions, account closures, regulatory filings, fund transfers
- Human approval required at every consequential action
- Agent prepares recommendation with full reasoning; human approves or rejects with documented rationale
- Dual approval for actions above financial threshold

Tier 4 — Human-Initiated Only (Critical Risk, Irreversible):

- Examples: large wire transfers, regulatory submissions, trade execution above limit
- Agent can prepare but cannot submit without explicit human initiation

Governance infrastructure: workflow engine (ServiceNow/Jira), approval SLA tracking, escalation on breach, full audit trail of every approval and rejection.

FOLLOW-UP CHALLENGE

Regulators require you to demonstrate that a specific AI credit decision was made by a human, not an AI. How does your architecture provide this evidence?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

SECTION 11

RELIABILITY ENGINEERING

[AI-Specific SLOs] [Global Outage Recovery] [Cross-Region Resiliency]

Q1 **PRINCIPAL** [SLOs] [AI Systems] [Observability]

How do you define AI-specific SLOs? What is different about measuring reliability for AI systems vs. traditional software?

DISCUSSION HINTS

- Non-determinism
- Model quality as reliability
- Latency distribution
- Token throughput

MODEL ANSWER

Traditional SLOs measure availability and latency. AI systems require additional dimensions:

Standard SLOs (also required for AI):

- Availability: 99.9% (3-9s) for non-critical AI, 99.95% for customer-facing
- Latency P50/P95/P99: LLM inference has different latency profiles — measure time-to-first-token (TTFT) and time-per-output-token (TPOT) separately
- Error rate: 4xx (client), 5xx (infrastructure), and model-specific errors (context limit exceeded, content filter triggered)

AI-specific SLOs:

- Quality SLO: % of responses meeting quality threshold (human evaluation sample + automated LLM-as-judge)
- Groundedness SLO: for RAG applications — % of responses grounded in retrieved context (hallucination rate inverse)
- Fairness SLO: for decision models — demographic parity ratio must remain within $\pm X\%$ over rolling 30 days
- Model drift SLO: input distribution drift detector — alert when feature distribution shifts $> Z$ sigma
- Cost SLO: token cost per transaction must not exceed target — cost is an operational reliability concern

Observability stack for AI: traces (LangSmith/Langfuse), metrics (Prometheus), evaluation platform (Ragas/Phoenix), cost dashboard (per model, per team).

FOLLOW-UP CHALLENGE

Your LLM quality SLO shows a sudden 20% drop in groundedness. Walk through your diagnosis.

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q2 **DISTINGUISHED** [Outage] [Recovery] [BCP]

Design a global outage recovery strategy for enterprise AI systems during a major cloud provider failure.

DISCUSSION HINTS

- Multi-provider failover
- Stateful vs. stateless AI
- RTO/RPO for AI
- Business continuity

MODEL ANSWER

AI systems have unique BCP challenges: model weights, vector stores, conversation history, and ongoing agent sessions.

Classification of AI workloads by recovery priority:

- Tier 1 (P0, <15 min RTO): customer-facing AI in active transactions, fraud detection, payment processing AI
- Tier 2 (P1, <2 hr RTO): internal productivity AI, underwriting AI, customer service agents
- Tier 3 (P2, <24 hr RTO): analytics AI, model training, experimentation workloads

Architecture for Tier 1 resilience:

- Multi-provider AI gateway: primary provider (Azure OpenAI) with automatic failover to secondary (AWS Bedrock)
- Model parity: same model capability available on both providers; tested monthly
- Stateless inference: no inference state stored in provider infrastructure
- Vector store replication: pgvector replicated cross-region and cross-cloud for RAG applications
- Conversation state: stored in application layer (Redis Cluster), not in model provider

During outage:

- Gateway automatically routes to secondary provider when health check fails (3 consecutive failures = failover)
- Agent sessions: graceful degradation — in-flight sessions suspended and resumed on recovery
- Communication: status page updated within 10 minutes; affected users notified

Chaos engineering: quarterly provider failover drills, annual full BCP simulation.

FOLLOW-UP CHALLENGE

After recovery, you discover that 4 hours of agent session state was lost. How do you communicate this to customers and what architectural change prevents recurrence?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

SECTION 12

BANKING SCENARIOS & REGULATED INDUSTRIES

[Global Banking AI Platform] [Multi-Jurisdictional Compliance] [Identity Convergence]

Q1 **DISTINGUISHED** [Banking AI] [Global Platform] [Design]

Design a global banking AI platform that must support RBI, PCI-DSS, GDPR, and MAS TRM requirements simultaneously.

DISCUSSION HINTS

- Regulatory requirement matrix
- Data residency by jurisdiction
- Unified audit
- Sovereign AI

MODEL ANSWER

Multi-jurisdictional AI architecture requires a layered compliance model.

Regulatory Requirement Matrix:

- RBI (India): payment data exclusively in India; audit access to infrastructure; no offshore processing of sensitive data
- PCI-DSS: cardholder data environment (CDE) isolated; all AI trained on payment data must be in CDE scope; tokenisation required
- GDPR: data minimisation for AI training; right to explanation for AI decisions; consent management; EU residency
- MAS TRM: concentration risk limits; outsourcing controls; exit planning; technology risk management governance

Architecture response:

Regional AI Plane design:

- Each jurisdiction gets a sovereign AI plane: compute, model hosting, vector store all within borders
- Models trained globally on anonymised/synthetic data, then fine-tuned within regional plane on local data
- AI Gateway deployed per region — no prompts or responses cross jurisdictional boundaries

Unified Governance Plane (metadata only — no regulated data):

- Model registry: global catalog of models with jurisdiction-specific deployment status
- Audit aggregation: compliance evidence collected per region, aggregated for Group-level reporting
- Policy distribution: responsible AI policies distributed globally; jurisdiction-specific overrides applied at region

PCI-DSS AI compliance:

- No cardholder data in prompts — tokenise before AI ingestion; de-tokenise in secure compute enclave after inference
- CDE AI workloads in dedicated, network-isolated AI cluster with PCI-DSS scoped infrastructure

FOLLOW-UP CHALLENGE

Your global model shows measurably different fraud detection accuracy for Indian customers vs. UK customers. How do you identify the cause and remediate?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs

- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q2 **CHIEF ARCHITECT** [Identity Convergence] [Customer] [Employee]

How would you architect customer and employee identity convergence for a global bank?

DISCUSSION HINTS

- *Progressive trust elevation*
- *Customer-employee boundary*
- *Privileged customer scenarios*
- *Staff customer accounts*

MODEL ANSWER

Identity convergence does not mean merging the systems — it means enabling controlled, audited transitions between identity domains.

Key scenarios requiring convergence:

1. Staff banking: employees who are also customers (bank staff with personal accounts)
2. Relationship managers: employees accessing customer systems with customer-tier access for service purposes
3. Business customers: corporate users who need both employee-like system access and customer-like portal access

Architecture approach:

- Keep workforce IdP (Entra ID) and CIAM platform separate at the infrastructure layer
- Implement identity federation bridge: workforce identity can assert customer identity claims via a controlled linking service
- Staff banking: employee links their workforce identity to their personal customer account — an explicit, audited, consent-based relationship stored in the CIAM
- All cross-domain access is: explicitly authorised, step-up authenticated, fully audited

Controls:

- Relationship managers accessing customer systems use a dedicated service role (not their personal customer identity)
- Staff accounts flagged in CIAM — enhanced monitoring for insider threat
- Regulatory requirement: staff banking access treated as privileged — all access logged to immutable audit store

The golden rule: a customer identity must never automatically inherit employee privileges, and an employee accessing customer systems does so under their employee identity with appropriate authorisation.

FOLLOW-UP CHALLENGE

A bank employee attempts to access their own loan file through the internal banking system rather than the customer portal. How does your architecture detect and respond?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications

- Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

SECTION 13

PRINCIPAL ARCHITECT CHALLENGE ROUND

[Multi-Cloud Defence] [Autonomous Operations] [Core Banking Modernisation]

Q1 **DISTINGUISHED** [Multi-Cloud] [CTO Defence] [Tradeoffs]

Defend a multi-cloud strategy to a CTO who argues it adds complexity without proportionate benefit.

DISCUSSION HINTS

- When multi-cloud is worth it
- When it isn't
- Strategic optionality
- Complexity cost honest assessment

MODEL ANSWER

The honest answer: the CTO is right in many cases. Multi-cloud IS more complex, and complexity has a real cost.

When multi-cloud is NOT justified:

- When the driver is 'avoiding lock-in' as an abstract principle — portability costs more than staying
- When teams lack cloud-agnostic engineering capability to maintain abstraction layers
- When the organisation is too small to staff multiple cloud centres of excellence

When multi-cloud IS justified:

- Regulatory requirement: sovereignty laws mandate specific data stay in specific clouds/regions
- Differentiated capability: genuinely superior services on different clouds for different workloads
- Resilience: DR strategy requires independence from a single provider's failure domain
- Commercial: competitive pricing leverage is real for organisations spending >\$10M/year

My actual recommendation to the CTO:

Start with a primary cloud for 80% of workloads. Add secondary cloud only where you have a specific, justified reason. Maintain cloud-agnostic practices (open data formats, container portability, federated identity) as options, not obligations. Reassess annually.

The best multi-cloud strategy is intentional, not ideological.

FOLLOW-UP CHALLENGE

The CTO asks: 'If we needed to exit Azure in 12 months, how long would it actually take and what would it cost?'

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

Q2 **DISTINGUISHED** [Core Banking] [Modernisation] [Legacy]

Modernise a legacy mainframe core banking system serving 20M customers with zero downtime tolerance.

DISCUSSION HINTS

- *Strangler fig at scale*
- *Event sourcing migration*
- *Dual-write pattern*
- *Rollback strategy*

MODEL ANSWER

This is one of the hardest problems in enterprise architecture. The mainframe is reliable — the danger is not replacing it, it is doing so incorrectly.

Strategy: Strangler Fig with Event Sourcing Migration — never a big bang.

Phase 1 — Observation and Instrumentation (6 months):

- Instrument every mainframe transaction. Understand exact load, transaction patterns, data dependencies.
- Implement Change Data Capture (CDC) on mainframe — every write event published to event stream
- Build shadow systems: new platform replays events in shadow mode, results compared but not committed

Phase 2 — Peripheral Capability Migration (months 6-18):

- Migrate non-core capabilities first: statements, notifications, reporting queries
- Mainframe remains system of record; new platform serves read traffic only
- Validate data consistency between systems continuously

Phase 3 — Dual-Write Pattern (months 18-30):

- New transactions written to both mainframe and new platform simultaneously
- New platform becomes system of record for new accounts
- Existing accounts migrate incrementally by cohort (lowest-risk accounts first)

Phase 4 — Cutover by Cohort (months 30-48):

- Cohort-by-cohort migration: 1% → 5% → 25% → 100%
- Each cohort has a defined rollback trigger — automatic revert if error rate exceeds threshold
- Mainframe maintained as hot standby until 12 months after full cutover

Risk management: designate a Chief Migration Risk Officer. Every phase has a published go/no-go criterion.

FOLLOW-UP CHALLENGE

During Phase 3 dual-write, you detect a 0.003% data inconsistency between mainframe and new platform. What do you do?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

SECTION 14

DISTINGUISHED ARCHITECT ROUND

[First Principles] [Why Things Fail] [Architecture Philosophy]

Q1 **DISTINGUISHED** [Transformation] [Failure Patterns] [First Principles]

Why do enterprise transformations fail? Answer from first principles, not from a framework.

DISCUSSION HINTS

- Systems thinking
- Incentive structures
- Complexity accumulation
- Org change vs. tech change

MODEL ANSWER

Enterprise transformations fail because they are fundamentally change management problems misdiagnosed as technology problems.

First principles analysis:

1. Complexity is the enemy, and transformation adds complexity before reducing it.

During migration, you run two systems simultaneously. Complexity doubles. The organisation's capacity to absorb complexity is finite. Transformations that underestimate this collapse under their own weight.

2. Incentives are misaligned. The people responsible for old systems have career equity in them. Transformation threatens their expertise value. Without explicit incentive realignment, the organisation unconsciously resists the transformation it officially sponsors.

3. The feedback loop is too long. Benefits appear in Year 3. Costs appear in Year 1. Without interim value milestones, the programme loses sponsorship before delivering.

4. Architecture is changed without changing the organisation. Conway's Law guarantees the new architecture replicates the old org structure.

5. Ambition exceeds capacity. Transformations are scoped for the aspirational organisation, not the actual one. The gap between them is where programmes die.

The fix: smaller scope, faster value realisation, explicit incentive alignment, team topology changes before system changes, and relentless complexity reduction before addition.

FOLLOW-UP CHALLENGE

You are hired to rescue a 3-year transformation programme that is 2 years in and has delivered 15% of its scope. What do you do in the first 30 days?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls

Q2 **DISTINGUISHED** [Governance] [Bureaucracy] [First Principles]

Why does governance become bureaucracy, and how do you prevent it?

DISCUSSION HINTS

- *Parkinson's Law*
- *Risk aversion asymmetry*
- *Governance as defensive behaviour*
- *Automation*

MODEL ANSWER

Governance becomes bureaucracy when the cost of the governance mechanism exceeds the cost of the risk it was designed to prevent.

The mechanism:

1. Governance is created in response to a real failure. The failure creates pressure. The committee is created. The process is documented.
2. The failure is not repeated. The governance gets credit. The process is preserved.
3. New requirements are added. The committee expands. The process grows. The failure it was designed to prevent fades from memory.
4. The governance now costs more than the original failure cost. But because the cost is distributed and invisible (engineer time lost to process), and the failure it prevents is hypothetical, the bureaucracy persists.
5. Risk aversion asymmetry: a governance body is never blamed for saying no. It IS blamed if it approves something that later fails. This creates structural risk aversion: the committee defaults to no, more review, more documentation.

Prevention:

- Automate governance: machines enforce rules without self-preservation bias
- Sunset clauses: every governance process expires unless actively renewed with evidence of value
- Measure the cost of governance, not just the cost of failures it prevents
- Invert accountability: governance bodies should be measured on the velocity they enable, not just the risks they catch

FOLLOW-UP CHALLENGE

You have inherited a governance model with 47 separate approval committees. What is your 90-day consolidation plan?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
 - Architecture principles and tradeoffs
 - Security and compliance implications
 - Operational and reliability considerations
 - Financial impact and platform economics
 - Governance, risk and regulatory controls
 - Alternative approaches and decision rationale
-

Q3 **DISTINGUISHED** [Platform Teams] [Bottleneck] [First Principles]

Why do platform teams become bottlenecks, and how do you architect a platform that cannot become one?

DISCUSSION HINTS

- Product thinking
- API-first
- Self-service by design
- Cognitive load caps

MODEL ANSWER

Platform teams become bottlenecks through a predictable pattern: they start as enablers and become gatekeepers.

The mechanism:

1. Platform team is created to reduce duplication and cognitive load. They build shared infrastructure.
2. Shared infrastructure needs to be changed. Requests flow to the platform team. The team has limited capacity. Queue builds.
3. Platform team, overwhelmed, adds process to manage demand. Jira tickets. Approval gates. Planning cycles. The process protects the team but frustrates consumers.
4. Stream-aligned teams route around the platform team: shadow infrastructure, exception requests, custom solutions. The platform loses relevance while the team maintains the illusion of control.

Architecture for a self-service platform that cannot become a bottleneck:

1. API-first: every platform capability is a callable API. The platform team's job is to maintain the API, not to execute requests on behalf of consumers.
2. Self-service by design: any consumer can provision any standard capability without a ticket. The platform team is involved only for non-standard requests.
3. Explicit interaction modes (Team Topologies): X-as-a-Service is the dominant mode. Collaboration mode is time-bounded for new capability development, then transitions to self-service.
4. Thinnest viable platform: resist the temptation to centralise everything. Only centralise what is genuinely cheaper and better centralised. Centralising mediocre capabilities is worse than not centralising at all.

FOLLOW-UP CHALLENGE

Your platform team has a 6-month backlog. Two senior engineers just quit. What do you do this week?

EXPECTED DISCUSSION AREAS

- Business drivers and outcomes
- Architecture principles and tradeoffs
- Security and compliance implications
- Operational and reliability considerations
- Financial impact and platform economics
- Governance, risk and regulatory controls
- Alternative approaches and decision rationale

QUICK REFERENCE — FRAMEWORK DECISION GUIDE

Decision	Choose	When
EA Framework	TOGAF	Methodology, governance, ADM process
EA Framework	Zachman	Classification taxonomy, audit traceability
Authorization	RBAC	Role-based, simple, high-volume, predictable
Authorization	ABAC	Context-sensitive, data-centric, regulatory
Authorization	ReBAC	Relationship-driven, fine-grained resource access
Identity	Entra ID	Workforce identity, M365 ecosystem, OIDC/SAML
Identity	CIAM Platform	Customer identity (50M+ scale), progressive profiling
Machine Identity	SPIFFE/SPIRE	Workload-to-workload, multi-cloud, Kubernetes
Secrets	HashiCorp Vault	Multi-cloud, dynamic credentials, unified audit
Data Architecture	Data Mesh	Domain maturity, product culture, decentralisation
Data Architecture	Data Fabric	Legacy estate, unified governance, no reorg
Cloud Strategy	Single Cloud	<50 engineers, no sovereignty req, speed priority
Cloud Strategy	Multi-Cloud	Differentiated capability, sovereignty, DR independence
Agent Oversight	Autonomous (T1)	Low risk, reversible, read-only actions
Agent Oversight	Human-in-Loop (T3)	Consequential decisions, financial impact, regulated

QUICK REFERENCE — BANKING REGULATIONS

Regulation	Jurisdiction	Key Architecture Requirements
SR 11-7	USA (Fed)	Model risk management: validation, tiering, documentation, monitoring
DORA	EU	Operational resilience, ICT vendor risk, incident reporting <4h, resilience testing
GDPR	EU	Data minimisation, residency, right to erasure, AI explainability
PCI-DSS	Global	CDE isolation, tokenisation, encryption, access control, audit logs
RBI Guidelines	India	Data localisation, audit access, no offshore payment data processing
MAS TRM	Singapore	Concentration risk limits, outsourcing controls, exit planning
BCBS 239	Global Banks	Risk data aggregation, accuracy, integrity, timeliness of reporting
SOC 2 Type II	USA	Security, availability, processing integrity, confidentiality, privacy
ISO 27001	Global	ISMS, risk assessment, controls framework, continuous improvement

ENTERPRISE ARCHITECTURE INTERVIEW HANDBOOK

Distinguished Architect Edition

Comprehensive preparation for Enterprise Architect, Principal Architect, Chief Architect, Distinguished Architect, Head of Architecture, CTO, and Chief AI Architect interviews.